Trade Desk: the architecture

This is the trunk. It describes the whole desk as it should be built. Every other document is a leaf of one of its sections, and every leaf starts by naming the section it belongs to.

The design is not invented from nothing. It is built on the desk that runs today, which is documented in full under legacy/. The old desk holds what the new one must keep: every card field and its source, every calculation, the research method, the O'Neil method, the journal's design. Where the old desk failed, the failure is named, and the new design says what prevents it.

How to read it. Sections 1 to 5 give the whole picture in one pass: purpose, the outside world, the parts, the data. Section 6 takes each domain in turn, always in the same shape. Sections 7 to 11 cover the day as it runs, the rules that apply everywhere, deployment, the migration, and the decisions left to you.

Contents

1. What the desk is for
2. The desk and the outside world
3. The ten domains
4. The running parts
5. The data architecture
6. Each domain
7. A trading day, end to end
8. Rules that apply everywhere
9. Deployment, backup and restore
10. From today's desk to this one
11. Decisions that are yours
12. The document map
13. Words used here

---

1. What the desk is for

One trader, one small server, a phone and a laptop. The desk does six jobs:

#	Job	The question it answers
1	**Find**	which stocks are worth looking at right now, by each of my methods?
2	**Know**	everything about each of those stocks, on one card: price structure, volume, float, short interest, borrow, news and catalyst, market and sector context, industry group, the full CANSLIM reading
3	**Read the market**	should I be trading at all today, which way, and where is the leadership?
4	**Learn**	which methods find stocks that move, and which cards are likeliest to move?
5	**Test**	if I traded this rule on the stocks my methods found, on the days they found them, what would have happened?
6	**Review**	what did I actually trade, and how did it go?

Job 2, **the card**, is the largest part of the desk by far. It draws on ten external sources and about twenty processing steps. The architecture is built around it.

The research method (kept exactly)

The desk is a research instrument before it is a trading screen. Its method, learned on the live desk and documented in legacy/screeners/METHODOLOGY.md, does not change:

- **A tool is one method.** Each tool has its own screeners, its own photo time, its own history and its own model. Tools never learn from each other.
- **One photo a day per tool (R1).** At the tool's capture minute, every card is frozen exactly as known then. Only photographed cards are training data.
- **Success is movement, measured in ATR.** From the entry minute to the close: `upR = (highest high − entry) ÷ ATR14`, `downR = (entry − lowest low) ÷ ATR14`. A good move is 1.3 ATR or more. There is no stop, target or exit in this measure: it judges what a screener finds, not a trade.
- **Two entries, A and B**, a few minutes apart, always kept separate.
- **A benchmark** (Stocks in Play) that every screener must beat, and a **mirror** for every screener to test whether its direction is real.
- **Shared facts are labels, never filters.** Market and sector context, O'Neil data and the cross-tool shortlist inform a card but never change which cards a tool finds.

What the desk does not do

It places no orders. An earlier live trading engine was removed from the code. Its leftovers in the journal are recorded in legacy/journal/ and are not part of this design.

---

2. The desk and the outside world



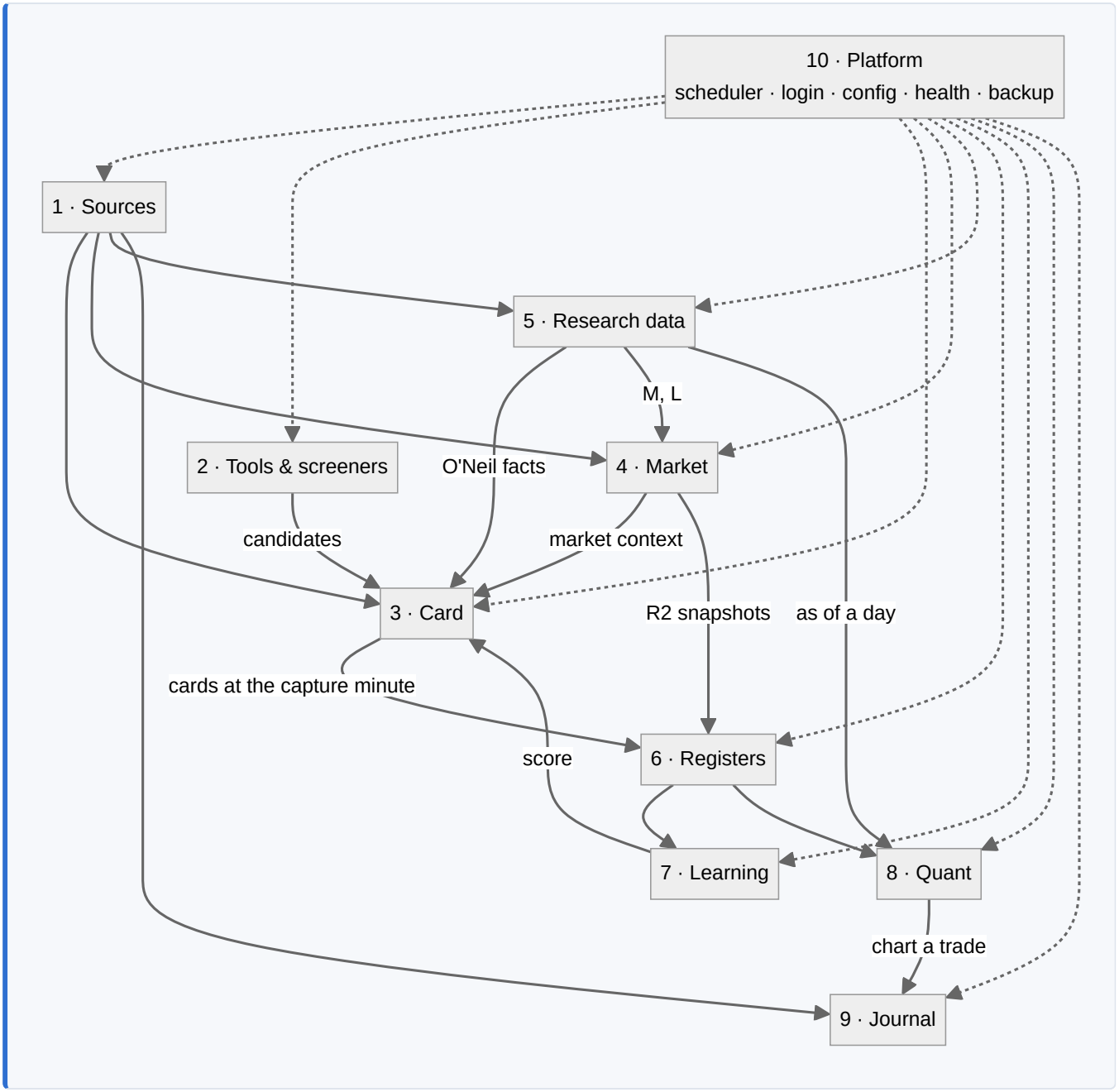
Source	What the desk takes from it	How often	Needs
TradingView scanner	candidates per screener, with 32 columns each; re-quotes; the index and sector-ETF quotes for the market read	every scan, every 5 minutes	nothing
TradingView news	headlines	live cards	nothing
Yahoo Finance	news; short interest (fallback); daily bars of ^GSPC and ^IXIC; 1-minute and daily bars for outcomes, charts and the journal	per scan, nightly, after the close, on demand	nothing
Alpaca	Benzinga news; the shortable flag; 1-minute and daily bars (fallback); the trader’s account fills	per scan, daily, after the close, on a schedule	key
FINRA	short interest: one file covering every symbol	daily	nothing
Nasdaq	short interest (fallback); Reg SHO threshold list	daily, per settlement day	nothing
NYSE	Reg SHO threshold list	per settlement day	nothing
SEC EDGAR	filings as news; company facts (earnings, sales, shares); SIC industry codes; Form 13F holdings	per scan, nightly, quarterly	a User-Agent
Polygon	grouped daily bars for the whole market (the RS rating); bars	nightly, on demand	key
Finnhub, **Google News**	news (Finnhub with a key; Google only when the others return fewer than 2 items)	live cards	key / nothing
Broker files	trades the trader uploads	on demand	—

Every source sits behind one adapter (§6.1).

3. The ten domains

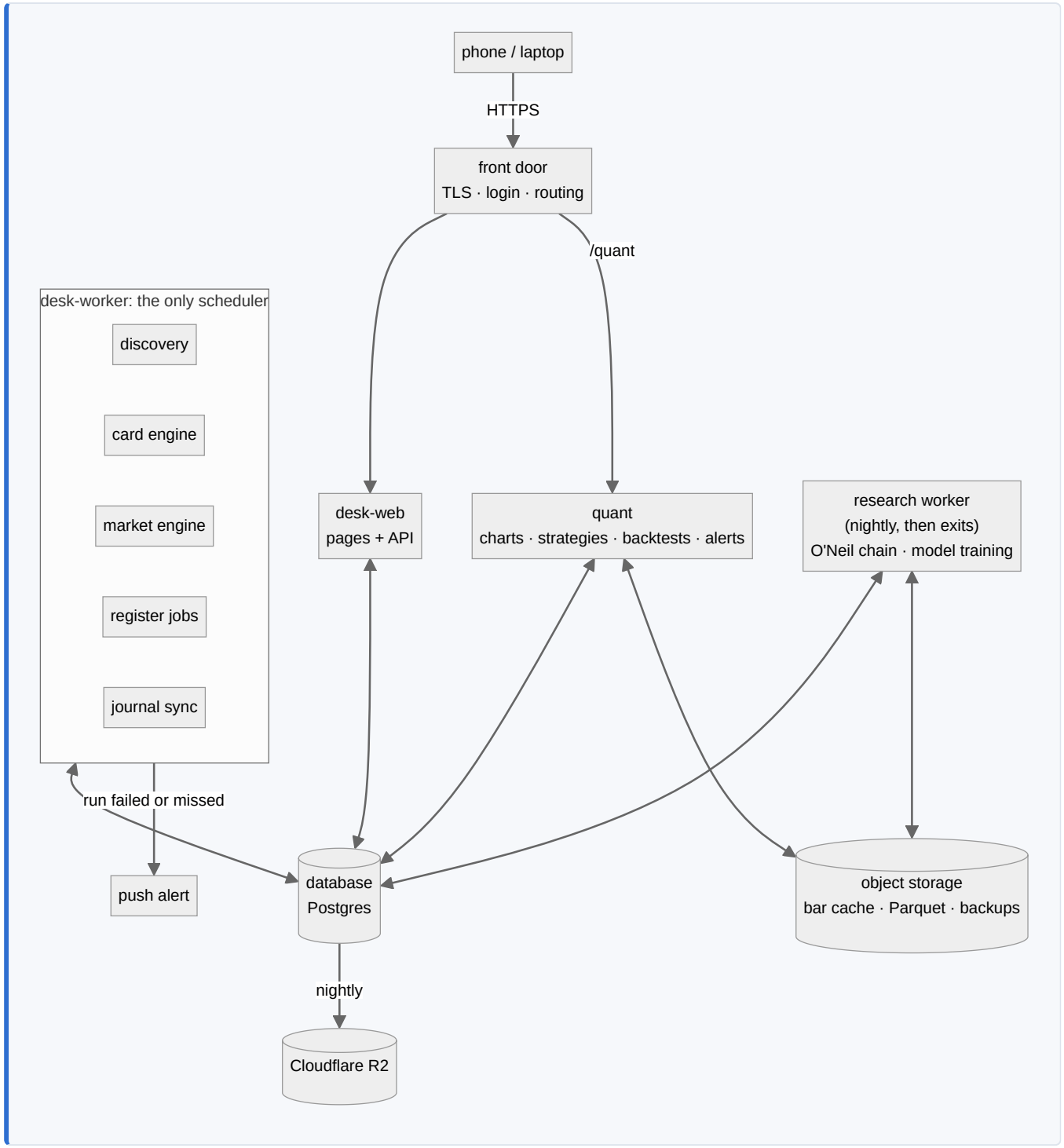
A domain is one area of responsibility with one owner. Each owns its data; the others read it.

#	Domain	Responsible for	Owns
1	**Sources**	talking to every external source: request, parse, rate, cache, judge health	source health, raw snapshots
2	**Tools and screeners**	what each tool hunts for, when, and running the hunt (discovery)	tools, screener versions, windows, capture times
3	**The card**	turning a candidate into a card: every fact, from every source, with its time	ticker facts, cards, per-tool live lists
4	**The market**	the market read: indexes, regime, sectors, hot sectors, and O’Neil’s M and L	market state and its history
5	**Research data**	the slow, heavy facts: the whole O’Neil chain (C, A, I, S, L, M, RS, groups, base, ratings), the SIC map	O’Neil facts with their dates
6	**Registers**	the research record: R1 photos, R2 market snapshots, R3 outcomes, R4 training rows, the shortlist	registers
7	**Learning**	the score per card, and the scorecard that judges screeners	models, scorecards
8	**Quant**	the verified indicator library, charts, strategies, backtests, live signal alerts, print sheets	strategies, backtests, the library
9	**Journal**	every real trade: broker fills and files, paired into trades, measured	trades, fills, accounts
10	**Platform**	the front door and login, the one scheduler, configuration, health and alerts, backup and restore, deployment, the page shell	users, job runs, settings



4. The running parts

Five programs and one database. Today every running tool is its own process with its own copy of the pipeline and its own scorer process, beside the archive, qp, the journal and the backup.


phone / laptop
HTTPS
front door
TLS · login · routing
/quant
desk-worker: the only scheduler
discovery
card engine
market engine
register jobs
journal sync
desk-web
pages + API
quant
charts · strategies · backtests · alerts
research worker
(nightly, then exits)
O'Neil chain · model training
run failed or missed
push alert
database
Postgres
object storage
bar cache · Parquet · backups
nightly
Cloudflare R2

Part	Language	Does	Never does
front door	Cloudflare Tunnel or Caddy	one HTTPS address, the login, routing to desk-web and quant	—
desk-web	Node (TypeScript)	every page and the API: the tools, cards, Market tab, registers, scorecard, settings, journal	run a timed job
desk-worker	Node (TypeScript), same code base as desk-web	the only scheduler. Hosts discovery, the card engine, the market engine, register jobs, journal sync. Records every run	serve pages
research worker	Python	the O'Neil chain and model training, after the close and on demand. Writes facts, then exits	stay running
quant	Python (FastAPI)	today's qp: the verified library, charts, strategies, backtests, live signal alerts, print sheets. Reads registers and facts from the database, as of any date	compute card or market facts
database	Postgres	the system of record	—
object storage	local disk, mirrored to R2	bar caches, Parquet exports, backups	—

Why two Node programs from one code base. The page server must stay quick and must never stop because a scan is slow, and the scheduler must never run twice. One code base keeps the card and market logic in one place; two processes keep their duties apart.

Why the research worker is separate. Today the O'Neil chain lives inside the chart server. A chart restart therefore blanks the CANSLIM reading on every card, and every scan of every tool calls the chart server. O'Neil results are card fields and Market-tab sections, so their writer must not depend on the chart being up (§6.5).

Why Postgres. One writer per table, real transactions, `as of` queries, and one backup instead of fifteen SQLite files. SQLite with one writer per file would also work on this box. What matters is the ownership rules, not the engine; the choice is yours (§11).

---

5. The data architecture

5.1 Three kinds of data

Kind	Example	Rule
Facts about the world	AAPL's short interest; SPY's close; NVDA's quarterly EPS	shared by every tool; stored **once**, with the time it became known and its source
Results of a method	T2 found AAPL at 09:41; T2's photo; T2's model	belong to one tool; never shared, never mixed
Records of the trader	strategies, backtests, trades, notes, settings	belong to the trader; edited only by the trader

5.2 The fact store

Every fact the desk knows about a ticker or the market is one row:

Column	Meaning
`subject`	a ticker (`AAPL`), an index, a sector ETF, or `market`
`family`	the fact family: `quote` , `short_interest` , `borrow` , `news` , `catalyst` , `oneil_c` , `rs_rating` , … (each family is defined in its domain's leaf)
`value`	the fields of that family, as JSON
`status`	`ok` , `none` (the source answered: there is nothing) or `unavailable` (could not ask), with a reason
`source`	which adapter produced it
`known_at`	when the desk learned it (UTC instant)
`as_of`	the date or instant the fact describes (a settlement date, a quarter end)

From this table:

- **the card** is the latest `ok` fact of each family at a given instant;
- **R1** is the card at the capture minute, frozen;
- **a backtest** reads the facts known before its decision bar, so nothing looks ahead;
- **two sources for one fact** are both stored, with a stated priority, never overwriting each other.

This one table replaces the per-process caches, the shared JSON files and the "as of" reconstructions of today's desk.

5.3 The main entities

Entity	Owner domain	One row per
tool	tools & screeners	tool (T1, T2, …)
screener version	tools & screeners	screener × version; R1 records which version it froze
fact	card, market, research data	subject × family × `known_at`
card	card	ticker × trading day (shared by every tool)
tool card	card	tool × ticker × trading day: screener keys, first seen, star, score, bias
market state	market	instant (each scan), with history
register row: R1, R2, R3, R4, shortlist	registers	tool × ticker × day (R2: tool × slot × day)
model, scorecard	learning	tool × version
strategy, backtest	quant	as today
account, fill, trade	journal	broker fill; round-trip trade
job, job run	platform	job; each run with its result
user, session, setting	platform	—

The full schema is a leaf: `design/DATA-MODEL.md` (§12).

5.4 Ownership: one writer for every table

Data	Only writer	Today it lives in
tools, screeners, windows, capture times	desk-web (checked on save)	`tools.config.json`, `tools.local.json`, each tool's SQLite
ticker facts	the card engine	per-process caches, card JSON, per tool
cards, tool cards	the card engine	each tool's memory, checkpointed
market state	the market engine	each tool's memory; hot-sector state lost on restart
O'Neil facts, SIC map	the research worker	`data/oneil-*.json`, `industry-map.json` (two writers: F-21), qp caches
registers	register jobs	one SQLite per tool
models, scorecards	the research worker	`src/scoring/outputs*` (not backed up)
strategies, backtests	quant	`~/.qp/platform.db`
trades, fills, accounts	the journal	`data/journal/journal.db` (with 21 leftover tables)
job runs	desk-worker	memory, per tool
secrets	the trader, in one secret file or store	`data/*.json` files, mode 0600

5.5 Time

- Every instant is stored in **UTC**. Every trading day is a **New York date**. Every clock time shown is **New York time**.
- One **trading calendar** (sessions, holidays, half days) serves every domain. Today each part guesses weekdays and ignores holidays, and three journal faults came from clock strings written as UTC or a fixed −05:00 offset (legacy/journal/KNOWN-ISSUES.md, D-J8, D-J10, D-J12).

---

6. Each domain

Every domain below has the same shape:

- **Job**: what it is responsible for.
- **Built on**: what the old desk did and meant to do, with the legacy documents that hold the detail.

- **What failed**: the faults found in the 2026-10 review, by their IDs in the legacy known-issues files.
- **The design**: how it is built here.
- **Leaf**: the design document with the full detail.

6.1 Sources

Job. Every call to the outside world. One adapter per source.

Built on. Today each source is called from inside the stage that needs it, in every tool's process. The calls themselves are well understood: endpoints, columns, fallbacks and quirks are all recorded in legacy/screeners/DATA-PIPELINE.md and legacy/qp/DATA-FEEDS.md. The data check (legacy/screeners/DATACHECK.md) already judges each source by what it returns rather than by whether it answered.

What failed.

- The same ticker's news, short interest and borrow are fetched once per tool that found it.
- A missing value once read as "no" or "bearish" (D-S1, D-S2, F-14).
- A column shift could misalign every field until the alignment check was added (D-S3).
- Source logic is scattered, so a fallback order exists in one stage and not in another.

The design. An adapter states, for its source:

It states	Example (short interest)
the fact families it produces	`short_interest`
how fresh a fact must be	one day
its rate limits and timeouts	FINRA: one file per day; Yahoo: per symbol, cookie and crumb
its honest null	`unavailable` with the reason; never 0
its place in the priority order for each family	FINRA → Yahoo → Nasdaq
how its health is judged	the share of symbols answered with a plausible value

The card engine, the market engine, the research worker and the journal ask adapters for facts; they never call a source directly.

Leaf: `design/SOURCES.md`.

6.2 Tools and screeners

Job. What each tool hunts for, when it may add cards, when it takes its photo, and running the hunt (discovery).

Built on. Eleven tools are registered, five running (T1, T2, T6, T7, T11), six archived but still served for backtests. A screener is a set of TradingView scanner rules with an order, a size, a run window, a check window and a mirror; the tradability floor (price ≥ $1, average volume ≥ 1M, ATR ≥ $1, ATR ≥ 3 % of price) is added to every screener at scan time. Each tool's screeners and why they are what they are: legacy/screeners/METHODOLOGY.md. The builder, its 52 fields, 15 operators, validation and mirror rules: legacy/screeners/SCREENER-BUILDER.md. Adding and stopping tools: legacy/screeners/TOOL-LIFECYCLE.md. The intent: legacy/screeners/IDEAS.md, part 2.

What failed.

- Each tool is a whole copy of the program, about 90 MB each; tools were stopped for lack of memory.
- T6 photographs at 10:16 a screener that starts at 11:30 (D-S9): times and windows are never checked against each other.
- A changed screener mixes two experiments under one name: there are no versions.
- Tests a scanner cannot express (T6's two-hour move, no news, six-month trend) run as a hidden gate inside discovery.

The design.

- **A tool is a row**: name, screeners, capture minute, entries A and B, status (running, paused, archived). One discovery engine runs every running tool; adding a tool adds rows, not a process.
- **A screener is versioned.** Saving a change creates a new version; R1 records the version of every screener that matched.
- **Checked on save**: the capture minute must fall after at least one of the tool's run windows has opened; refused otherwise, with the reason.
- **Post-scan tests are named steps of the screener** (move rate, no news, trend), visible in the builder and in the scan report.
- **Discovery** runs each due screener once per scan, merges the matches into one candidate per ticker per tool, and hands new tickers to the card engine.

Leaf: `design/TOOLS-AND-SCREENERS.md`.

6.3 The card

Job. Everything known about a ticker, on one card, each fact with its source and time. The largest domain.

Built on. The card's every field, its source and when a blank is legitimate: legacy/screeners/CARD-FIELDS.md. Its layout and display rules: legacy/screeners/CARD.md. The twelve pipeline stages that fill it today: legacy/screeners/DATA-PIPELINE.md. The CANSLIM fold's exact specification: legacy/qp/CANSLIM.md §7 and §11.

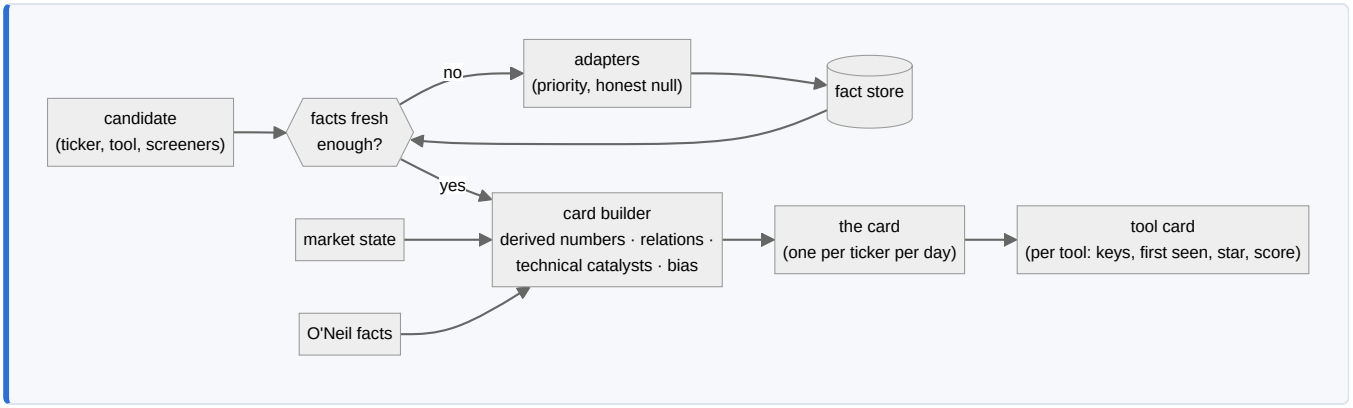
The card today, by fact family:

Fact family	Sources, in priority	Fresh for	On the card
quote, levels, ranges, volume, float, cap (32 columns)	TradingView scanner	5 minutes	header, Volume & Float, Range, Price, EMAs
derived numbers and relations (gap, ADR %, range positions, distances, MA stack, PM/ADR)	computed from the quote	with the quote	signal tags, PM/ADR, range bars
short interest, days to cover	FINRA → Yahoo → Nasdaq	a day	Volume & Float
borrow: shortable, easy to borrow	Alpaca	a day	borrow note
Reg SHO threshold	Nasdaq + NYSE lists	per settlement day	borrow note
news	TradingView, Alpaca (Benzinga), Yahoo, EDGAR, Finnhub; Google as fallback	each scan, live cards	News fold
catalyst (39 categories, 3 tiers), technical catalysts, auto bias	computed from news and the tape	with the news	badges, bias
market context (regime, trend stages, sector bias, hot sector, themes)	the market engine	each scan	Market Context fold
industry, group	SIC (EDGAR), TradingView label only where SIC has none	nightly	header, L
CANSLIM: C, A, N (base), S, L, I, M, RS, U/D, A/D, RS line	the research worker	nightly, plus per scan for M	CANSLIM fold, `cs*` columns
score	the tool’s model	at discovery	score
shortlist marks	this tool; every tool	live	☆, ★ also

What failed.

- Every tool builds its own copy of a ticker’s card.
- While qp restarts, every card’s CANSLIM fold is blank, and every scan freezes empty `cs*` columns into R1.
- Facts carry no time, so R1 cannot say when a fact was learned, and backtests rebuild CANSLIM “as of” a day.
- R4 carries fewer columns than R1 (D-D2).

The design.



- **One card per ticker per day**, shared by every tool. A tool card adds only what belongs to the tool: which screeners matched, when first seen, the star, the score, the manual bias.
- **Each fact family refreshes on its own clock**: the quote every 5 minutes for live cards; news every scan; short interest daily; O'Neil nightly.
- **A failed source never fails the card**; its family shows `unavailable` with the reason.
- **Every field the old card shows is kept**, with its formula and its display rule. The leaf lists them one by one.

Leaf: `design/CARD.md`.

6.4 The market

Job. The market read, built once for every tool, kept with its history, and stamped on every card.

Built on. The Market tab today has two halves (legacy/screeners/MARKET-TAB.md):

Section	Built from	Today
Indexes	TradingView: SPY, QQQ, DIA, IWM, VIX	each tool, each scan
Desk analysis: short-term bias, mid-term stage, long-term bias, the regime	index columns: SMAs 5/20/50/200, Bollinger, ADX, VWAP, relative volume	each tool, each scan
Sector heatmap and hot sectors	15 sector ETFs	each tool, each scan; hot state in memory
M: O'Neil market model	^GSPC and ^IXIC daily bars: distribution days, stalling days, rally attempts, follow-through	qp, nightly
L: industry groups	the SIC map and whole-market RS ratings	qp, nightly

What failed.

- Five tools compute the same snapshot from the same ETFs, minutes apart.

- Hot-sector state is lost on every restart.
- Sector bias is lopsided: the score runs about −39 to +100 (F-16).
- The mid-term stage counts two signals twice (D-D5).
- R2's O'Neil columns are read at export, so an old day shows today's status (D-D3).
- With no index data, the desk once read the market as bearish (D-S2).

The design.

- **One market engine** builds the market state each scan, for every tool, and stores it with history. R2 is a view of that history at the tool's slots, frozen with whatever O'Neil status was current then.
- **The desk's read and O'Neil's method stay separate** on the tab and in the data. The first answers "what is the tape doing now", the second "should you be buying at all".
- **Hot-sector state is counted per trading day and persisted.**
- **No regime without data**: if SPY and QQQ are both incomplete, there is no regime, said plainly.
- Sector score centred on 0 (−100 to +100), and the mid-term stage counting distinct facts; both need re-tuning, recorded in the leaf.

Leaf: `design/MARKET.md`.

6.5 Research data: the O'Neil chain

Job. The slow, heavy facts, computed after the close: everything O'Neil's CANSLIM needs, plus the SIC industry map.

Built on. The O'Neil method as built, with its exact rules, traps, thresholds and the MarketSmith-style panels it reproduces from free data: legacy/qp/CANSLIM.md (1,646 lines: the deepest document of the old desk). The nightly job, its six steps and how it is installed: legacy/qp/OPERATIONS.md.

Letter	Computed from
C, **A**: quarterly and annual earnings, sales, acceleration, stability, ROE	SEC EDGAR company facts
N: the base (weeks, depth, pivot, handle, % to pivot)	weekly bars
S: supply, U/D volume ratio, A/D letter	EDGAR shares; daily bars
L: RS rating, group rank, RS in group	Polygon grouped daily (whole market); the SIC map
I: fund holders, change, direction	Form 13F
M: market model state, distribution days, follow-through	^GSPC, ^IXIC daily bars

What failed.

- It runs inside the chart server; a restart blanks every card’s CANSLIM reading.
- Every scan of every tool calls the chart server for each card’s values.
- The nightly job runs from a systemd timer installed by hand; without it, the O’Neil data goes stale with no alarm (Q-N5).
- The industry map had two writers and flipped every morning (F-21).
- When an O’Neil file is missing, the Market tab asks the chart server to build it inside a page request.

The design.

- **The research worker** runs the chain after the close (and on demand). It writes O’Neil facts into the fact store with their `as_of` dates, then exits. It is the **only** writer of those facts and of the SIC map.
- **The card and the Market tab only read.** No page request ever computes O’Neil data.
- **A run that fails or does not happen raises an alert** (platform scheduler), so stale data is never silent.
- **The backtest’s CANSLIM gate reads the same facts as of the day**, so live and backtest use one source.
- **The method does not change**: every rule and threshold in the legacy document carries over.

Leaf: `design/ONEIL.md`.

6.6 Registers

Job. The research record: what each tool saw and what happened next.

Built on. legacy/screeners/REGISTERS.md, with every column:

Register	Written	Holds
R0	continuously	today’s live list (in the new design: the tool cards)
R1	the tool’s capture minute	every card of today, frozen whole; the only training data
R2	8 slots, 09:26–10:01	the market state around the photo
R3A / R3B	16:05	from entry A / B: entry price, highest high, lowest low, ATR14 (from days before), upR, downR, opening range
R4A / R4B	16:05	R1 joined to R3: the training rows
Shortlist	each star, and the auto-rule one minute before R1	the day’s picks, manual and automatic

What failed.

- R4 is narrower than R1: no CANSLIM columns, no week, quarter or year ranges (D-D2).
- The opening range is stored but not exported (D-D4).
- R1's price is called `price936` in R4 whatever the capture time (P-1).
- Outcomes use weekdays, not a trading calendar.

The design.

- **R1 freezes the tool card and the card at the capture minute**, plus the screener versions and the market state id. Nothing is lost: every field of the card is in the photo.
- **R4 is a view** over R1 and R3, carrying every R1 column; no copy that can drift.
- **R3 runs on the trading calendar**, and records a reason for every ticker it could not measure.
- **Exports are generated from the schema**, so a new column appears in every export at once.

Leaf: `design/REGISTERS.md`.

6.7 Learning: the score and the scorecard

Job. For each card: of the stocks this tool found before that looked like this one, how many made a good move? For each screener: keep it, watch it, or drop it?

Built on. The intent, rules and open questions: legacy/screeners/IDEAS.md, parts 1 and 3. Today's score (PCA, five buckets, six bases, regime sub-tables) and scorecard, step by step, and why neither should be copied: legacy/screeners/SCORING-TODAY.md.

What failed.

- The scorer is off on every tool, so no card has a score, and the shortlist auto-rule adds nothing (D-D1).
- Five scorer processes, one per tool, each holding its tables in memory.
- The scorecard grades in one direction only (D-S12).

The design.

- **Trained by the research worker** after the outcomes are in, one model per tool, from that tool's history only. A model is a versioned record in the database.
- **Applied by the card engine at discovery** from the stored model. No scorer process.
- **Six questions, scored separately**: entry A or B × long, short or either. The card shows the one its bias points to.
- **Honest about thin data**: every score carries its sample size and confidence; below a minimum, no score.
- **Explainable**: which features moved it, and how the most similar past cards did.
- **The scorecard** grades each screener against its tool's benchmark and its own mirror, on the share of good moves and the share of days hit, with the sample shown beside every verdict.

Leaf: `design/LEARNING.md`.

6.8 Quant: charts, strategies, backtests, alerts

Job. Chart any stock with verified indicators; build trading rules by clicking; backtest them on the stocks the tools found, on the days they found them; watch today's lists for live signals; print sheets.

Built on. The whole of qp: legacy/qp/README.md and its fifteen documents. The verified library (71 primitives, each approved against TradingView) and its code: legacy/qp/LIBRARY.md, legacy/qp/library/. The strategy grammar, fill models and exit plan: legacy/qp/STRATEGIES.md. The backtest and its honesty rules: legacy/qp/BACKTEST.md.

What failed.

- Alerts evaluate with the `close` fill while backtests use `desk`, so an alert can fire on a different bar from the backtest's (Q-N9).
- `breakeven_after_target` is reported but never simulated (Q-N7).
- Some routes default to the Polygon feed, not the desk's default (Q-N6).
- It hosts the O'Neil chain (moved out: §6.5).

The design.

- **Quant keeps its library, engine and backtest as they are**; they are the best-built part of the desk.
- **One evaluator and one fill model** for preview, backtest and alerts.
- **It reads registers and facts from the database, as of a day**, instead of over HTTP from each tool's port.
- **It computes no card or market facts.**

Leaf: `design/QUANT.md`.

6.9 Journal

Job. Every real trade, after the fact: broker fills and uploaded files, paired into round trips, measured, reviewed.

Built on. legacy/journal/README.md and its eight documents: the page and every formula, the Alpaca accounts and their pairing, the file formats, the analysis layer, the data. The Alpaca pairing (running position, weighted prices, reversals split, stable ids) is the model to keep.

What failed. File imports mispair scale-ins and scale-outs, lose TradingView shorts and duplicate on re-import (D-J1 to D-J7). No trade has a stop, so every R is empty (D-J9). Times are clock strings, with a fixed −05:00 offset and one date per trade (D-J8, D-J10, D-J12). The technical snapshot and the grading half do not work (D-J13, D-J14).

The design.

- **One pairing engine** for every source: Alpaca fills, broker files, hand entry.

- **A trade has an entry instant, an exit instant, its fills, its fees and a stop.** The stop comes from the broker's stop order, the strategy's exit plan, or the trader.
- **Stable ids from the source's own facts**, so a second import updates.
- **The entry snapshot comes from the fact store** as of the entry minute, and the R1 row when the ticker was on a tool that day.
- **The chart button** opens quant at the trade's real instants.

Leaf: `design/JOURNAL.md`.

6.10 Platform

Job. Everything every domain relies on: the front door and login, the one scheduler, configuration, health and alerts, backup and restore, deployment, and the page shell.

Built on. The login shared across ports (`td_auth` cookie, `data/auth.json`); the jobs and the Monitor tab (legacy/screeners/MONITOR.md); settings (legacy/screeners/SETTINGS.md); the data check (legacy/screeners/DATACHECK.md); the nightly backup to R2 with daily, monthly and analysis copies; `deploy-tools.sh` under pm2; the shared app bar. Today's arrangement: legacy/architecture/ARCHITECTURE.md.

What failed.

- Thirteen public ports, and once no login at all (S-1).
- Jobs spread over five processes, pm2 cron and a hand-installed systemd timer, with no run history and no alert when one fails or does not run.
- Restore is a manual copy, file by file, with processes stopped by hand, and has never been rehearsed. Trained models and the keys are not in the backup.
- No CI: fixes are verified only when someone runs the tests.
- Two supervisors in the past caused a 113,854-restart incident; a qp restart took 3 min 20 s.

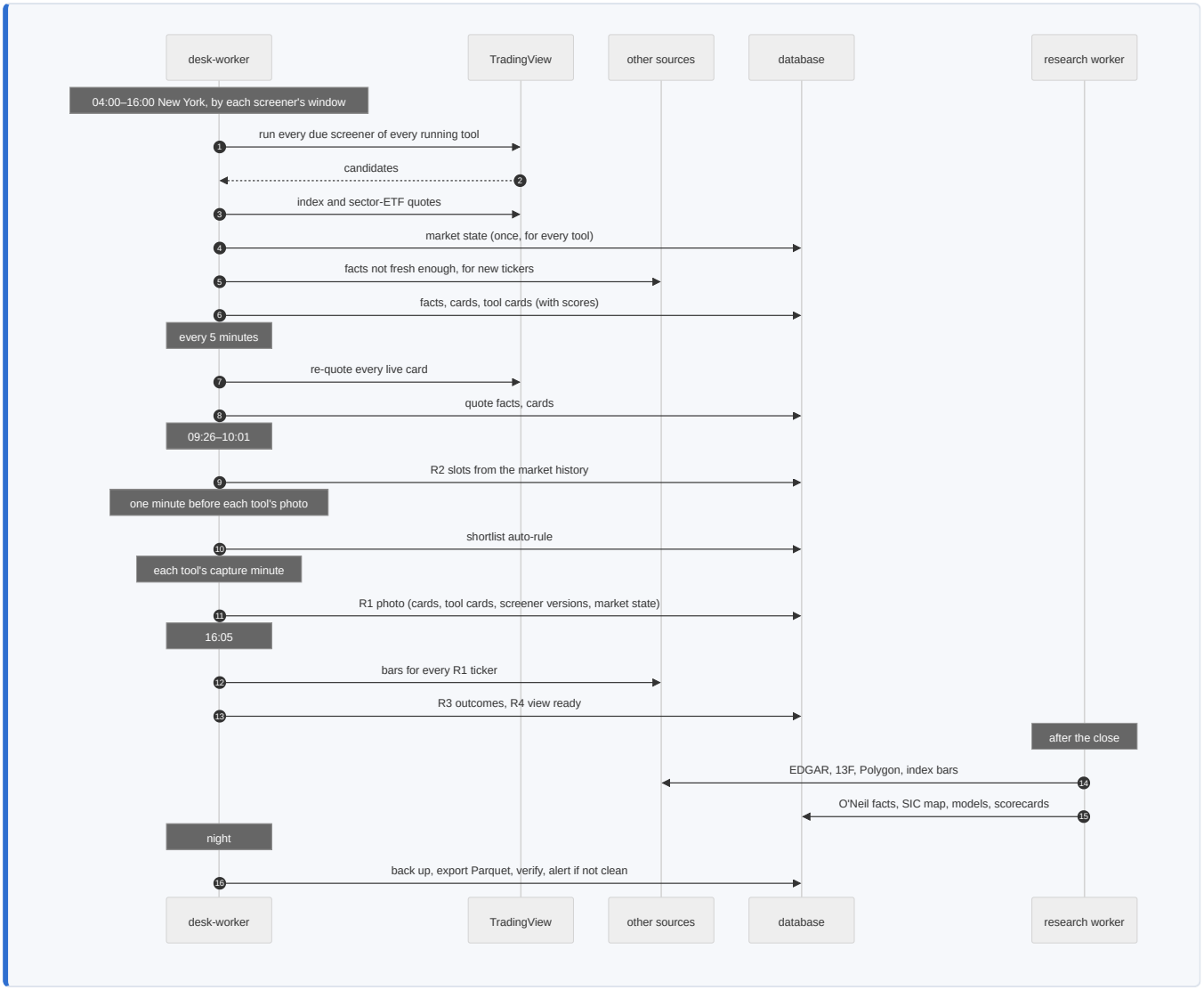
The design.

- **One front door**: one HTTPS address, one login, every other port closed.
- **One scheduler** (desk-worker) with every job as a row: its schedule, its last runs, their results. A run that fails, or does not happen when it should, sends a push alert.
- **Configuration checked on save**, with the reason for any refusal.
- **Health** is the data check made permanent: each source and each domain reports green, amber or red, on one page.
- **Backup**: the database and Parquet exports every night, verified; **restore is one command**, rehearsed monthly onto a spare database.
- **CI** runs every test on every pull request; red blocks the merge.
- **One supervisor** (systemd or pm2, never both).

Leaf: `design/PLATFORM.md`.

---

7. A trading day, end to end



Time (New York)	Job	Domain
04:00–09:00 every 30 min; 09:00–10:00 every 5; 10:00–16:00 every 15; 16:00	discovery scans (each screener only inside its window)	tools & screeners, card, market
04:00–16:00 every 5 min	re-quote live cards	card
09:26, 09:31 … 10:01	R2 market snapshots	registers
each tool’s capture minute − 1	shortlist auto-rule	registers
each tool’s capture minute (09:36 … 10:16)	R1 photo	registers
16:05	R3 outcomes, R4	registers
after the close	O’Neil chain, SIC, training, scorecards	research data, learning
on each account’s schedule	journal sync	journal
night	backup, verify	platform

---

8. Rules that apply everywhere

These rules come from faults found on the live desk. Each leaf applies them to its domain.

Rule	Means	Learned from
Honest null	a missing value is `unavailable` or `none` with a reason; never 0, never bearish, never “no”	D-S1, D-S2, F-14
No look-ahead	every computed value uses only what was known at its instant; bars after an entry never reach that entry’s snapshot	F-20, D-J13
A label, never a filter	shared facts never change which cards a tool finds	the research method
One writer per fact	every table and fact family names its single owner; two sources are both stored, with a priority	F-21
Isolate results, share facts	a tool’s results, model and history never reach another tool	the research method
One definition per number	each formula is written once, in one place, and named in its leaf	D-J17, Q-N9
Versioned definitions	a screener, a check or a model that changes gets a new version, and history records which one ran	screener mixing, the check library
Checked on save	configuration that cannot work is refused when saved, with the reason	D-S9
Failures are loud	a job that fails or does not run sends an alert; a silent no-op is a bug	Q-N5, the first snapshot-id repair
Proven before it ships	every change runs the tests in CI; every source and file format has fixtures	the untested journal importer
Secrets stay put	keys live in one secret file or store, mode 0600, never in a reply, a log, git or a backup	S-2 (keys pushed to GitHub)
Real instants	UTC stored, New York shown, one trading calendar	D-J8, D-J10, D-J12

9. Deployment, backup and restore

The box. One small Linux server (today an AWS EC2 instance, about 660–740 MB used by the desk’s processes). The target uses about 480–600 MB for five tools, and each extra tool costs a few MB instead of about 90.

Process	Supervised as	Memory (estimate)
desk-web	service	~150 MB
desk-worker	service	~200 MB
quant	service	70–150 MB
database	service	60–100 MB
research worker	timer job; exits when done	runs, then 0
front door	service	~20 MB

Deploy. CI builds and tests every pull request. On merge, one command on the box pulls, migrates the database, restarts the services in order and checks each one's health page; it stops and says why at the first failure.

Backup. Every night: a database dump and the Parquet exports to R2, verified by restoring the dump into a scratch database and counting rows. The daily, monthly and analysis layout of today's backup is kept (legacy detail in the code repo's `docs/BACKUPS.md`, to be carried into the platform leaf).

Restore. One command: `restore <date>` stops the services, restores the database, starts the services, and checks health. A monthly rehearsal restores last night's backup to a spare database and reports the row counts. Keys are restored from the trader's own secret copy, never from the backup.

---

10. From today's desk to this one

Each step ships on its own, keeps the desk working, and can be undone.

#	Step	Domain	Risk
1	CI on every pull request	platform	none
2	One front door; close every other port	platform	low · needs a domain name
3	Configuration checked on save	tools & screeners	low
4	The trading calendar and real instants everywhere	platform	low
5	The fact store and source adapters; tools read short interest, borrow and news from it	sources, card	medium · compare cards old and new for a week
6	The market engine, once for all tools; hot state persisted	market	low–medium
7	The research worker: the O’Neil chain leaves the chart server	research data	medium
8	One scheduler with run history and alerts	platform	medium · run side by side for a week
9	Tools as rows: one engine for every tool; R1 compared row by row before each old process stops	tools & screeners, card, registers	the largest step · reversible per tool
10	The score and scorecard rebuilt	learning	medium
11	The journal’s one pairing engine, real instants, stops	journal	medium · re-import and compare
12	Postgres and Parquet; one restore command, rehearsed	platform	medium · the backup proves the copy

The full plan, with what each step changes and how it is checked: `design/MIGRATION.md`.

---

11. Decisions that are yours

Decision	Options	Recommendation
Where it runs	stay on EC2; a managed host	stay: the design fits one small box
The database	Postgres; SQLite with one writer per file	Postgres, for transactions, `as of` queries and one backup
A domain name	needed for one HTTPS address	yes
How far to go	steps 1–8 remove most duplicated work and risk; step 9 removes the per-tool memory cost	all, in order
Push alerts	ntfy, Pushover, Telegram	any that reaches your phone

12. The document map

Every document has a place in this tree. The design leaves are written one by one; the legacy documents are complete.

Trunk section	Design leaf	Legacy evidence it builds on
6.1 Sources	`design/SOURCES.md` (to write)	DATA-PIPELINE, DATA-FEEDS, DATACHECK
6.2 Tools and screeners	`design/TOOLS-AND-SCREENERS.md` (to write)	METHODOLOGY, SCREENER-BUILDER, TOOL-LIFECYCLE, IDEAS part 2, PAGES
6.3 The card	`design/CARD.md` (to write)	CARD-FIELDS, CARD, DATA-PIPELINE, CANSLIM §7, §11
6.4 The market	`design/MARKET.md` (to write)	MARKET-TAB, CANSLIM §2, §8, §9
6.5 Research data	`design/ONEIL.md` (to write)	CANSLIM, qp OPERATIONS
6.6 Registers	`design/REGISTERS.md` (to write)	REGISTERS, METHODOLOGY
6.7 Learning	`design/LEARNING.md` (to write)	IDEAS parts 1 and 3, SCORING-TODAY
6.8 Quant	`design/QUANT.md` (to write)	qp: LIBRARY, CATALOG, STRATEGIES, BACKTEST, ALERTS-AND-PRINTS, PAGE, SCALPS, PINE
6.9 Journal	`design/JOURNAL.md` (to write)	journal: PAGE, ACCOUNTS, IMPORT, ANALYSIS
6.10 Platform	`design/PLATFORM.md` (to write)	today’s architecture, MONITOR, SETTINGS, DATACHECK
5 Data architecture	`design/DATA-MODEL.md` (to write)	the three legacy DATA-MODEL documents
all	`design/API.md` (to write)	the three legacy API documents
10 Migration	`design/MIGRATION.md` (to write)	the earlier rebuild draft

The open findings of the old desk, by program: screeners, qp, journal.

Each design leaf has the same five parts: its place in this trunk; the legacy design in full (every field, source and calculation it keeps); what failed; the new design; and its contracts (tables, facts, routes) with the other domains.

---

13. Words used here

Word	Meaning
tool	one screening method (T1 … T11), with its own screeners, photo and history
screener	a set of TradingView scanner rules, with a window, an order and a mirror
card	everything known about one ticker on one day
tool card	what one tool adds to a card: its screeners, first seen, star, score
fact	one known thing about a ticker or the market, with its source and time
fact family	a group of facts fetched and refreshed together (quote, short interest, news, …)
R0 … R4	the registers: live list, photo, market snapshots, outcomes, training rows
photo (R1)	every card of a tool, frozen at its capture minute
entry A / B	the two entry minutes after the photo at which outcomes are measured
upR / downR	how far a stock ran for / against, in ATR, from the entry to the close
good move	upR ≥ 1.3 ATR
benchmark	Stocks in Play: the screener every other must beat
mirror	the same screener facing the other way
tradability floor	price ≥ $1, average volume ≥ 1M, ATR ≥ $1, ATR ≥ 3 % of price, added to every screener
honest null	“unavailable, because …” instead of a made-up 0 or “no”
as of	the facts known at a given instant, and nothing later